

Object Detection from Cityscapes Using Custom CNN and YOLOv8 Evaluation

<https://github.com/ebrivera/ds340-final>

Ernesto Rivera, Sanjeev Pal

Faculty of Computing and Data Sciences

DS 340: Intro to Machine Learning and Artificial Intelligence

Professor Kevin Gold

April 30, 2025

Introduction

As part of this project, we aimed to create a model that is capable of detecting cars and buses in street images from the Cityscapes dataset. The majority of object detection demonstrations utilize datasets such as COCO that come with bounding boxes and were originally intended to be used in this type of application. Cityscapes exist to be used in a different way—semantic segmentation—so it doesn't natively come with bounding boxes. This required additional effort on our part to utilize the data in object detection.

From it, we wanted to train our own custom model to detect and classify cars and buses. In addition to that, we also wanted to compare against a pretrained model, yOLOv8 in order to gauge how good our custom model was in comparison. This enabled us to demonstrate that we were capable of creating a complete machine learning pipeline from the ground up and learn to compare and assess different models as well.

Methodology

For the purpose of this project, we implemented a model of object detection from scratch with the use of the Cityscapes dataset. Because the Cityscapes is designed for semantic segmentation and does not include bounding boxes, we needed to generate them ourselves. We implemented a function named `extract_largest_vehicle_bbox()` that identifies the largest object in each image that falls into one of five vehicle categories: car, truck, bus, train, and bicycle. This function determines bounding boxes by taking the upper-left and lower-right pixel coordinates that surround each object. This, however proved to be a fault in our program which we will discuss later on in the results. Next, we converted all images to 256×512 pixels and scaled the bounding box coordinates so that we could utilize them during training.

We had 2,000 training and 500 validation samples. Each was paired with its respective label, and we used the Cityscapes label scheme to map the original IDs to training IDs and discarded irrelevant pixels (pixels that were not a vehicle). These image-label pairs were structured into TensorFlow datasets and batched with `tf.data` to load them efficiently in batches. For training purposes, we employed a custom Generalized Intersection over Union (GIoU) (appendix e) loss to enhance the way that the models produced bounding boxes. GIoU is more than the simple overlap and beneficial in the case of boxes that do not overlap substantially. GIoU is a very common tool used when trying to figure out the efficiency of your computer vision models.

We developed and trained four CNN models. BaseDetector (appendix a) was the first, and most simple. It consisted of three convolutional pools using ReLU for the activation function followed by a global average pooling layer that leads into two dense heads. The reason that there are two dense heads is because we needed one head for classification of the object (vehicle type) and then we needed another head to find the bounding box of the object. This is a pattern that continues throughout the other models we trained. The second model, we called BNDetector

(appendix b), consisted of the same convolutional layers, however we included two dense layers in the bounding box classification because we were trying to hone in on that part and found we couldn't get accurate enough results. Additionally, we modified the head of the classification by adding a dropout to prevent overfitting. Then, we build a deeper model called DeeperDetector (appendix c), which is the same thing as the previous BNDetector except it has one extra convolutional layer on the main part of the architecture (before splitting into two heads). Finally, our last model was called AugDeeperDetector (appendix d), which is a modification of the DeeperDetector, except for that it includes augmentation at the beginning: horizontal flips, rotations, and also zooms. Like the previous architecture, it also includes two dense layers leading into the bounding box head and dropout on the classification. For the two heads of the CNN, we had a linear activation function on the bounding box head, and then we had a softmax activation function on the classification head.

All the models were trained with the Adam optimizer and classification was taken care of by use of `sparse_categorical_crossentropy` and bounding box regression with GIoU loss. The bounding box loss was also prioritized by providing it with five times more weight compared to the classification loss. This is because we found it harder to improve the bounding boxes as opposed to the actual classification (classification is a lot easier especially when you have so much samples). To prevent overfitting, we employed early stopping on the validation bounding box loss. Lastly, we wanted to compare our results to a SOTA model, YOLOv8, to see how we compared to it. For this experiment, we used a smaller sample since we weren't too interested in seeing the results, of 20 images in validation with a function that measured the Intersection over Union (IoU) between the actual and the predicted boxes.

Result

All four models trained successfully and achieved high accuracy in classification. The BaseDetector model trained to completion in 7 epochs with 91.14% accuracy. Training proceeded in a smooth manner and didn't demonstrate significant overfitting. The BNDetector was slightly better with a 91.51% accuracy, but plateaued slightly. The DeeperDetector was almost in the same range (91.57%), however, the additional layers didn't bring significant improvement. Finally, the AugDeeperDetector used a similar model except with augmentation and demonstrated 90.77% accuracy, and seemed to have been met with a bit of overfitting.

Then when looking at the Bounding Box Accuracy that we measured with our Intersection over Union measurement, which is a common measurement of the accuracy of tracked objects in Computer Vision models, we found very unsettling results. BaseDetector had a mean IoU of 30.4% with 31.2% of the pictures highlighting over 50% accurate, BNDetector demonstrated a mean IoU of 30.2% with an $\text{IoU} \geq 0.5$ of 31.2%, DeeperDetector 29.8% mean IoU with 31.2% of the results greater than 50% accurate, and finally AugDeeperDetector showing us the lowest detection accuracy of 31.2% over 50% IoU with the average at 28.5%. It wasn't until our final pass through that we realized why this happened, and it was quite disappointing to discover. We decided to continue throwing different dense layers onto our

bounding box head, but sometimes the problems are a lot simpler and require a close eye. On the last pass through, it was recognized that the `extract_largest_vehicle_bbox()` method wasn't working correctly, in fact it was working terribly. As previously mentioned, we were using a dataset that was split by polygonal annotation of dash cam footage. Each vehicle is classified into one of car, truck, bus, train, bicycle. However, what we failed to neglect is that sometimes there are more than one of the same type of vehicle on the road (especially in a situation where there is a lot of traffic). That being said, if there is a truck on the bottom left and another truck on the top right, then it will use the corners for the bottom left most part of the truck and the top right most part of the second truck. This sunk my heart looking at this because it explained the days we had spent analyzing why our classification was so low! SO, in the final hours we tried to use skimage measure (appendix f), which groups blobs together into different connectivity groups (think of the very common number of islands in LeetCode), however this wouldn't change the fact that most cars are still going to look like one massive blob in polygonal segmentation form when they're from the perspective of a driver.

As a point of comparison with industry standards, we also trained the YOLOv8n model with no fine-tuning. On the same validation set, the mean IoU of YOLOv8n was about 33.9%, which wasn't much higher than the results we witnessed with our own custom models. When we initially saw how bad our IoU numbers were from our initial four models, we decided it was best to see how State of the Art vision models performed, and this helped us look deeper into the code instead of just deciding to throw more layers at the problem. Nevertheless, YOLOv8n was pre-trained over the huge COCO dataset with millions of labels, whereas we trained with only 2,000 training samples and developed our labels in-house. Therefore, although our models fell behind that of YOLO in performance, the outcome was remarkable with limited resources.

Conclusion

This project was also able to demonstrate that it is feasible to create a detection pipeline that works from the ground up from a dataset such as Cityscapes that was not originally created specifically with this in mind. By translating segmentation masks into bounding boxes and creating four distinct CNN architectures, we trained models capable of classifying and localizing cars in images of cities. The BaseDetector was the best overall performer, most likely because it didn't start overfitting in both heads. Even though our models' bounding box accuracy didn't give us results that we were happy about, we learned the importance of making sure that you understand why your results aren't working the way you expect them to, debug consistently, and try to view why the detection is working the way it is. If we were to go back and fix anything about this assignment, it would be to ensure that we could figure out a more plausible solution to the bounding box problem, though this sparked our interest to learn more about how segmentation works.

All four models were consistently good, which indicates that our data pipeline and training setup were working optimally and that architecture decisions were good. Utilizing a

custom GIoU loss and focusing on bounding box loss during training proved to be a decent approach, particularly with limited images. We may be able to enhance performance in the future by using different types of labels or moreover cleaner data, applying transfer learning with pre-trained networks, or even retraining YOLOv8 on our custom data set. In general, working on the project assisted us in learning how to construct and test the entire object detection framework and also learn to work with actual data sets that aren't ideally in the right format.

Appendix a:

Model: "BaseDetector"

Layer (type)	Output Shape	Param #	Connected to
input_image (InputLayer)	(None, 256, 512, 3)	0	-
conv2d (Conv2D)	(None, 256, 512, 32)	896	input_image[0][0]
max_pooling2d (MaxPooling2D)	(None, 128, 256, 32)	0	conv2d[0][0]
conv2d_1 (Conv2D)	(None, 128, 256, 64)	18,496	max_pooling2d[0][0]
max_pooling2d_1 (MaxPooling2D)	(None, 64, 128, 64)	0	conv2d_1[0][0]
conv2d_2 (Conv2D)	(None, 64, 128, 128)	73,856	max_pooling2d_1[0][0]
global_average_pooling2d (GlobalAveragePooling2D)	(None, 128)	0	conv2d_2[0][0]
dense (Dense)	(None, 128)	16,512	global_average_pooling2d[0][0]
class_output (Dense)	(None, 5)	645	global_average_pooling2d[0][0]
bbox_output (Dense)	(None, 4)	516	dense[0][0]

Appendix b:

Model: "BNDetector"

Layer (type)	Output Shape	Param #	Connected to
input_image (InputLayer)	(None, 256, 512, 3)	0	-
conv2d_3 (Conv2D)	(None, 256, 512, 32)	896	input_image[0][0]
max_pooling2d_2 (MaxPooling2D)	(None, 128, 256, 32)	0	conv2d_3[0][0]
conv2d_4 (Conv2D)	(None, 128, 256, 64)	18,496	max_pooling2d_2[0][0]
max_pooling2d_3 (MaxPooling2D)	(None, 64, 128, 64)	0	conv2d_4[0][0]
conv2d_5 (Conv2D)	(None, 64, 128, 128)	73,856	max_pooling2d_3[0][0]
global_average_pooling2d (GlobalAveragePooling2D)	(None, 128)	0	conv2d_5[0][0]
dense_1 (Dense)	(None, 256)	33,024	global_average_pooling2d[0][0]
dropout (Dropout)	(None, 128)	0	global_average_pooling2d[0][0]
dense_2 (Dense)	(None, 128)	32,896	dense_1[0][0]
class_output (Dense)	(None, 5)	645	dropout[0][0]
bbox_output (Dense)	(None, 4)	516	dense_2[0][0]

Appendix c:

Model: "DeeperDetector"

Layer (type)	Output Shape	Param #	Connected to
input_image (InputLayer)	(None, 256, 512, 3)	0	-
conv2d_18 (Conv2D)	(None, 256, 512, 32)	896	input_image[0][0]
max_pooling2d_13 (MaxPooling2D)	(None, 128, 256, 32)	0	conv2d_18[0][0]
conv2d_19 (Conv2D)	(None, 128, 256, 64)	18,496	max_pooling2d_13...
max_pooling2d_14 (MaxPooling2D)	(None, 64, 128, 64)	0	conv2d_19[0][0]
conv2d_20 (Conv2D)	(None, 64, 128, 128)	73,856	max_pooling2d_14...
max_pooling2d_15 (MaxPooling2D)	(None, 32, 64, 128)	0	conv2d_20[0][0]
conv2d_21 (Conv2D)	(None, 32, 64, 256)	295,168	max_pooling2d_15...
global_average_pooling2d_10 (GlobalAveragePooling2D)	(None, 256)	0	conv2d_21[0][0]
dense_9 (Dense)	(None, 256)	65,792	global_average_p...
dropout_4 (Dropout)	(None, 256)	0	global_average_p...
dense_10 (Dense)	(None, 128)	32,896	dense_9[0][0]
class_output (Dense)	(None, 5)	1,285	dropout_4[0][0]
bbox_output (Dense)	(None, 4)	516	dense_10[0][0]

Appendix d:

Model: "AugDeeperDetector"

Layer (type)	Output Shape	Param #	Connected to
input_image (InputLayer)	(None, 256, 512, 3)	0	-
random_flip (RandomFlip)	(None, 256, 512, 3)	0	input_image[0][0]
random_rotation (RandomRotation)	(None, 256, 512, 3)	0	random_flip[0][0]
random_zoom (RandomZoom)	(None, 256, 512, 3)	0	random_rotation[...
conv2d_10 (Conv2D)	(None, 256, 512, 32)	896	random_zoom[0][0]
max_pooling2d_7 (MaxPooling2D)	(None, 128, 256, 32)	0	conv2d_10[0][0]
conv2d_11 (Conv2D)	(None, 128, 256, 64)	18,496	max_pooling2d_7[...
max_pooling2d_8 (MaxPooling2D)	(None, 64, 128, 64)	0	conv2d_11[0][0]
conv2d_12 (Conv2D)	(None, 64, 128, 128)	73,856	max_pooling2d_8[...
max_pooling2d_9 (MaxPooling2D)	(None, 32, 64, 128)	0	conv2d_12[0][0]
conv2d_13 (Conv2D)	(None, 32, 64, 256)	295,168	max_pooling2d_9[...
global_average_pooling2d_10 (GlobalAveragePooling2D)	(None, 256)	0	conv2d_13[0][0]
dense_5 (Dense)	(None, 256)	65,792	global_average_p...
dropout_2 (Dropout)	(None, 256)	0	global_average_p...
dense_6 (Dense)	(None, 128)	32,896	dense_5[0][0]
class_output (Dense)	(None, 5)	1,285	dropout_2[0][0]
bbox_output (Dense)	(None, 4)	516	dense_6[0][0]

Appendix e: <https://giou.stanford.edu/>

Appendix f: <https://scikit-image.org/docs/stable/api/skimimage.measure.html#>